

Advanced Data Access with JdbcClient

Chapter 5 — Pro Spring Boot 4 (4th ed) ·

chapters/chapter-05-jdbc-client/{customer, management}

Spring Boot 4.0.6

Spring Framework 7.0.7

JdbcClient (6.1+)

@Transactional

Maven + Gradle

WebFlux functional endpoints

Contents

1. [What this chapter covers](#)
2. [Code walkthrough](#)
3. [Mermaid diagrams](#)
4. [Live run & endpoint evidence](#)
5. [Session context](#)

1 · What this chapter covers

Chapter 5 refactors JDBC access onto the fluent `JdbcClient` API (Spring Framework 6.1+) and layers on transactions and multi-table modeling. Two modules illustrate it:

- `customer/` (Maven, servlet/Tomcat) — single-table CRUD via `JdbcClient`, named + positional parameters, automatic record mapping vs a custom `RowMapper`, and a portable update-then-insert upsert.
- `management/` (Gradle, reactive/WebFlux) — a four-table domain (company, customer, address, communication) with foreign keys, a `@Transactional` service that writes all tables atomically, and **functional endpoints** (`RouterFunction` / `HandlerFunction`).

2 · Code walkthrough

The JdbcClient fluent pipeline

CustomerRepository.java (customer)

Every data-access call is a builder chain:

```
jdbcClient.sql(...).param(...).query(Customer.class).optional().
```

`JdbcClient` is created from the autoconfigured `DataSource` and wraps

`JdbcTemplate` / `NamedParameterJdbcTemplate` behind one fluent surface.

Two parameter styles

CustomerRepository.java:31-48

`.param("id", id)` binds a **named** parameter (`:id` in SQL); `.params(a, b, c)` binds **positional** `?` parameters. The repository mixes both — named for lookups, positional for the upsert SQL.

Automatic record mapping vs custom RowMapper

CustomerRepository.java:19-59

`.query(Customer.class)` auto-maps columns to record components by name (zero boilerplate). `findAll()` instead passes an explicit lambda `RowMapper` to show the escape hatch for projections that don't match a record 1:1.

Database-agnostic update-then-insert

CustomerRepository.java:27-42

Rather than dialect-specific `ON CONFLICT / MERGE`, `save()` runs an `UPDATE`; if it affects `0` rows it falls back to `INSERT`. Portable across H2 and PostgreSQL.

Profiles: H2 (dev) vs PostgreSQL (prod)

application-dev.properties · application-prod.properties

`spring.profiles.active=dev` selects `jdbc:h2:mem:customers`; the `prod` profile targets `jdbc:postgresql://localhost:5432/customers`. `schema.sql` initializes the table in both.

Multi-table domain & FK schema

management/schema.sql · model/*.java

Four tables with foreign keys: `customer.company_id → company`, `address.customer_id → customer`, `communication.customer_id → customer`. Each is a record; repositories follow the same `JdbcClient` pattern. `CustomerDetailsDTO` aggregates them for reads.

Declarative transactions

ManagementService.java:36-64

`@Transactional createCustomerWithDetails(...)` writes `company → customer → address → communication` in one atomic unit (FK order respected); a failure rolls all four back. `@Transactional(readOnly = true) getCustomerDetails(...)` reads and assembles the DTO.

Reactive functional endpoints

ManagementConfiguration.java:48-51 · reactive/ManagementHandlers.java

Instead of `@RestController`, routes are declared as a `RouterFunction`: `route(GET("/api/v1/management/customers/{id}"), handlers::getCustomerDetails)`. The handler returns `Mono<ServerResponse>` — `ok().bodyValue(dto)` or `notFound()`.

3 · Mermaid diagrams

Rendered from Mermaid and embedded as self-contained images (they also print in the PDF export). Expand each for the re-renderable source.

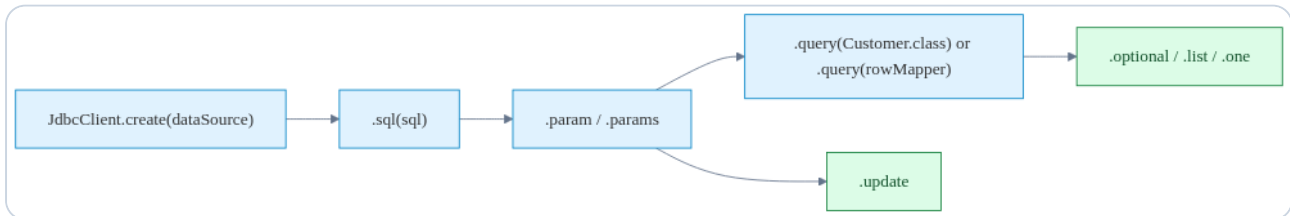


Fig 1 — JdbcClient fluent pipeline. Build a statement, bind parameters, then either read (query → optional/list/one) or write (update).

[Mermaid source](#)

```
%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line  
flowchart LR  
  A["JdbcClient.create(dataSource)"] --> B[".sql(sql)"]  
  B --> C[".param / .params"]  
  C --> D[".query(Customer.class) or .query(rowMapper)"]  
  D --> E["optional / .list / .one"]  
  C --> F["update"]  
  classDef term fill:#dcfce7,stroke:#15803d,color:#14532d;  
  class E,F term;
```

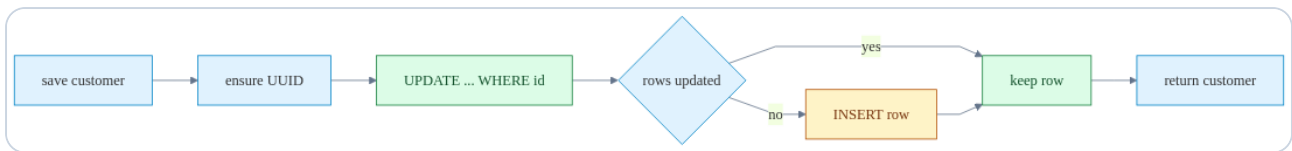


Fig 2 — Portable upsert. Try `UPDATE`; if zero rows changed, `INSERT`. Avoids dialect-specific syntax.

[Mermaid source](#)

```

%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
flowchart LR
    S["save customer"] --> ID["ensure UUID"]
    ID --> U["UPDATE ... WHERE id"]
    U --> D{"rows updated"}
    D -->|yes| RET["keep row"]
    D -->|no| I["INSERT row"]
    I --> RET
    RET --> END["return customer"]
    classDef upd fill:#dcfce7,stroke:#15803d,color:#14532d;
    classDef ins fill:#fef3c7,stroke:#b45309,color:#78350f;
    class U,RET upd;
    class I ins;
  
```

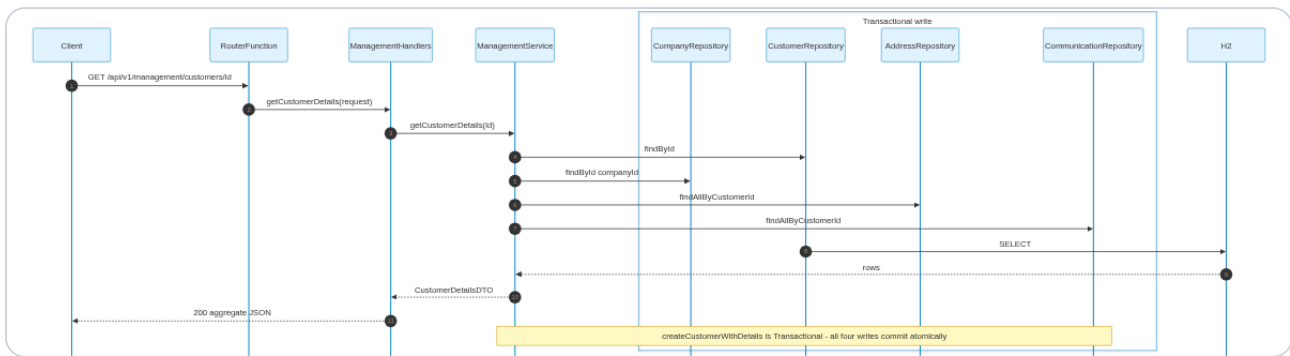


Fig 3 — Management request flow. A functional route delegates to the service; the read assembles an aggregate, while the write path is a single `@Transactional` unit across all four repositories.

Mermaid source

```

%%{init: {'theme':'base','themeVariables':{'primaryColor':'#e0f2fe','primaryBorderColor':'#0284c7','line
sequenceDiagram
    autonumber
    participant CL as Client
    participant RT as RouterFunction
    participant HN as ManagementHandlers
    participant SV as ManagementService
    box Transactional write
        participant CO as CompanyRepository
        participant CU as CustomerRepository
        participant AD as AddressRepository
        participant CM as CommunicationRepository
    end
    participant DB as H2
    CL->>RT: GET /api/v1/management/customers/id
    RT->>HN: getCustomerDetails(request)
    HN->>SV: getCustomerDetails(id)
    SV->>CU: findById
    SV->>CO: findById companyId
    SV->>AD: findAilByCustomerId
    SV->>CM: findAilByCustomerId
    CU->>DB: SELECT
    DB-->>SV: rows
    SV-->>HN: CustomerDetailsDTO
    HN-->>CL: 200 aggregate JSON
    Note over SV,CM: createCustomerWithDetails is Transactional - all four writes commit atomically

```

4 · Live run & endpoint evidence

Both modules were built and run; endpoints exercised with `curl`.

customer (Maven, Tomcat, H2)

```
The following 1 profile is active: "dev"
HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:customers user=SA
Tomcat started on port 8080 (http)
Started CustomerApplication in 36.91 seconds
# ApplicationReadyEvent seeds 3 rows via JdbcClient
```

Request	Result	Note
GET /api/v1/customers	200	Rows read via custom <code>RowMapper</code>
POST valid	200	Returns the entity directly (no <code>201</code> / <code>Location</code>)
POST invalid	400	<code>ProblemDetail</code> with <code>errors</code> as a Map
POST duplicate email	500	<code>UNIQUE</code> violation, unmapped by the advice

```
# POST invalid -> 400 application/problem+json
{"detail":"Validation failed","instance":"/api/v1/customers","status":400,"title":"Bad Request",
 "errors":{"name":"Name is required","email":"Email must be valid"}}
```

management (Gradle, Netty/WebFlux, H2)

```
Starting ManagementApplication using Java 21.0.12 # Gradle toolchain auto-detection reused the installed JDK
The following 1 profile is active: "dev"
HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:management user=SA
Netty started on port 8080 (http)
Started ManagementApplication in 16.039 seconds
Creating customer 1: CustomerDetailsDTO[customer=Customer[...Felipe Gomez...], company=Company[Apress...], ... (3 transactional seeds total)
```

Request	Result	Note
GET /api/v1/management/customers/{id}	200	Aggregate <code>CustomerDetailsDTO</code> from the functional route
GET unknown id	404	<code>ServerResponse.notFound()</code>
POST same path	404	Only a <code>GET</code> route is mapped

```
# GET .../customers/c6c3b8a4-... -> 200
{"customer":{"...Felipe Gomez...","companyId":"176d32c0-..."},
 "company":{"companyName":"Apress","industry":"Publishing",...},
 "addresses":[{"street":"123 Main St","city":"New York",...}],
 "communications":[{"communicationType":"email","communicationValue":"felipe.gomez@apress.com"}]}
```

5 · Session context

This document is the record of the working session that produced it.

Request

“explain code, run app and create mermaid diagrams in html file. session context should be captured in same html file” — for `chapters/chapter-05-jdbc-client/`.

Steps performed

1. **Explored** both modules (12 customer files, 18 management files) + docs.
2. **Explained** the code (Section 2) with file/line references.
3. **Ran customer** via `mvn spring-boot:run` (Tomcat/H2, dev profile) and exercised endpoints.
4. **Ran management** via `sh gradlew bootRun` — the wrapper downloaded Gradle 9.2.1; the toolchain spec then resolved to the machine's **already-installed JDK 21** (auto-detection, no JDK download). WebFlux/Netty booted and the `@Transactional` seeds ran.
5. **Generated** three Mermaid diagrams (fluent pipeline, upsert, request flow), rendered to PNG and embedded as data URIs.
6. **Authored** this self-contained HTML report.

Observations & suggested improvements

- **Blocking JDBC on a reactive stack:** `management` uses WebFlux/Netty, but `JdbcClient` is blocking — the handler runs JDBC on the event loop, which can starve it. Prefer a servlet stack for blocking JDBC, or offload to `Schedulers.boundedElastic()`.
- **Both apps default to port 8080** — they can't run together unless one is moved (`--server.port=8081`).
- **POST returns 200, not 201 + Location** in the customer module (returns the entity directly) — less RESTful than Chapter 3's controller.
- **DB errors leak as 500:** duplicate email (UNIQUE) isn't mapped; add an `@ExceptionHandler(DataIntegrityViolationException)` → `409 ProblemDetail`.
- **Upsert race:** update-then-insert isn't atomic without a unique key + transaction; concurrent inserts of the same id could both see 0 rows. Fine for the demo, worth noting.

Environment & tooling

- Runtime: Gradle daemon on Java 25; build toolchain resolved to the **installed JDK 21.0.12** (`/usr/lib/jvm/java-21-openjdk-arm64`) via auto-detection — nothing was downloaded (no foojay resolver configured). Spring Boot 4.0.6 / Spring 7.0.7; Tomcat 11 (customer) and Netty (management); HikariCP; H2 in-memory.
- Diagrams rendered via `mermaid.ink` and embedded as base64 data URIs — the HTML has **no external scripts, CSS, or image files**.
- Mermaid conventions: theme `%%{init}%%`, quoted labels, `box` grouping, `classDef` color-coding.

Reproduce

```
# customer (Maven, servlet)
cd chapters/chapter-05-jdbc-client/customer
mvn spring-boot:run

# management (Gradle, WebFlux)
cd chapters/chapter-05-jdbc-client/management
sh gradlew bootRun          # first run downloads Gradle 9.2.1; toolchain uses the installed JDK 21

curl http://localhost:8080/api/v1/management/customers/{id}
```

Chapter takeaways. `JdbcClient` unifies named/positional binding, automatic record mapping, and a fluent chain over the same pooled `DataSource`; the update-then-insert pattern keeps SQL portable; `@Transactional` makes multi-table writes atomic; and functional endpoints offer a lightweight alternative to annotated controllers — just keep blocking JDBC off the reactive event loop.

Generated from `chapters/chapter-05-jdbc-client/` · modules `customer/` (Maven) and `management/` (Gradle). Self-contained (no external resources). Companion to the Chapter 2–4 reports.